U.E 5.1 - Intelligence Artificielle : Machine learning
Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France



BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr
BENIGNO DA SILVA Matheus Henrique
matheus.benigno@ensta.fr

Sommaire

3 Examen des données, prétraitements et extraction des de scripteurs

Question 3.1: Compléter la fonction preprocessing() permettant de charger les deux images et de les afficher. indication: loadImages() .

Listing 1: Fonction de prétraitement

```
import numpy as np
import matplotlib.pyplot as plt

from loadImages import loadImages
from selectFeatureVectors import selectFeatureVectors
from displayFeatures2d import displayFeatures2d
from displayFeatures3d import displayFeatures3d


#%% def preprocessing
def preprocessing():

    img73, img87 = loadImages()

    fig = plt.figure(figsize=(10,6))
    plt.subplot(121)
    plt.imshow(img73)
    plt.title("Aral Sea - 1973")

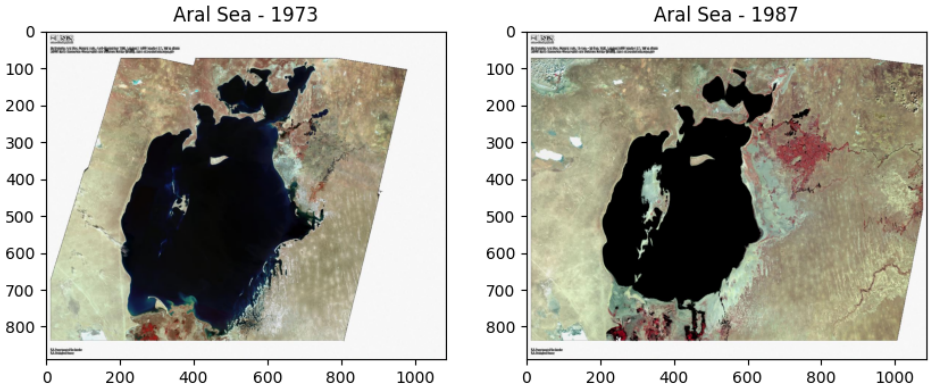
    plt.subplot(122)
    plt.imshow(img87)
    plt.title("Aral Sea - 1987")

    plt.show()

    featLearn, nbPix, nbFeat = selectFeatureVectors(img73, 10)

    return featLearn,img73,img87

if __name__ == "__main__":
    preprocessing()
```



Question 3.2: Que peut-on déduire de l'analyse de ces images ? Comment sont codées les valeurs des pixels ?

- L'analyse de ces images permetre de visualiser l'asséchement de la mer d'Aral. On observe une diminution de la surface d'eau bleue. Les images chargées sont de matrices 3D où chaque pixel est codé sur 3 canaux, ils sont rouge, vert et blueu.

Question 3.3: Les images sont en couleurs et nous allons utiliser les composantes des couleurs comme descripteurs de classification. Quelle est la dimension de l'espace des descripteurs lorsque l'image est codée en RGB (RVB) ?

- La dimension de l'espace des descripteurs est de 3. Chaque pixel est répresenté par un vecteur contenant trois valeurs : vec_pix = [rouge, vert, bleu].

Question 3.4: Tronquer de la même manière les deux images pour faire disparaître le texte dans la partie haute et basse des images.

Pour tronquer les deux images il suffit d'ajouter ces deux lignes de codes dans 1 :

```
1    margin = 50
2    im73 = im73[margin+10:-margin, :, :]
3    im87 = im87[margin+10:-margin, :, :]
```

4 Constitution d'un ensemble de données d'apprentissage

Question 4.1: Compléter la fonction preprocessing() par une partie qui vise à constituer une base d'apprentissage. Pour cela, réalisez un sous-échantillonnage de l'image de 1973 avec un pas de 500 points (indication : selectFeatureVectors.py). Quel est le nombre de données d'apprentissage?

On peut ajouter ces lignes de code :

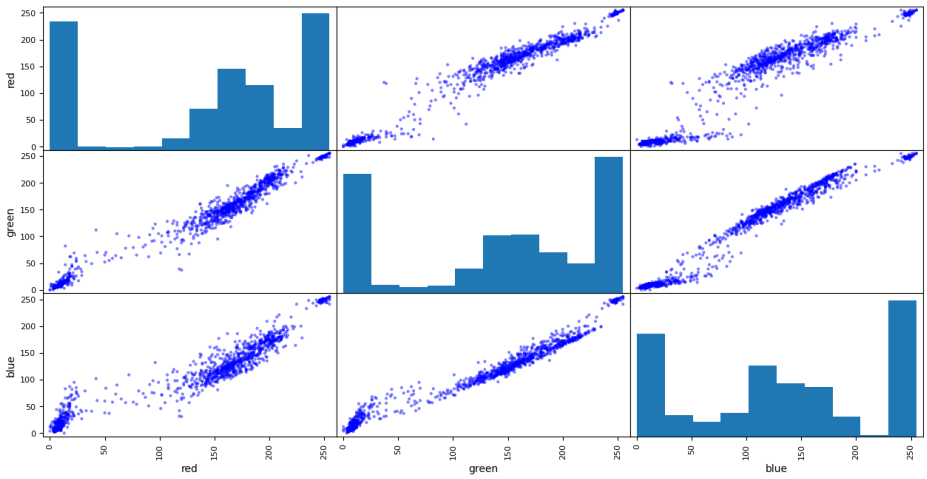
```
1    ...
2    featLearn, nbPix, nbFeat = selectFeatureVectors(img73,
         500)
3
4    print("Nombre de donnees dapprentissage : ", nbPix)
```

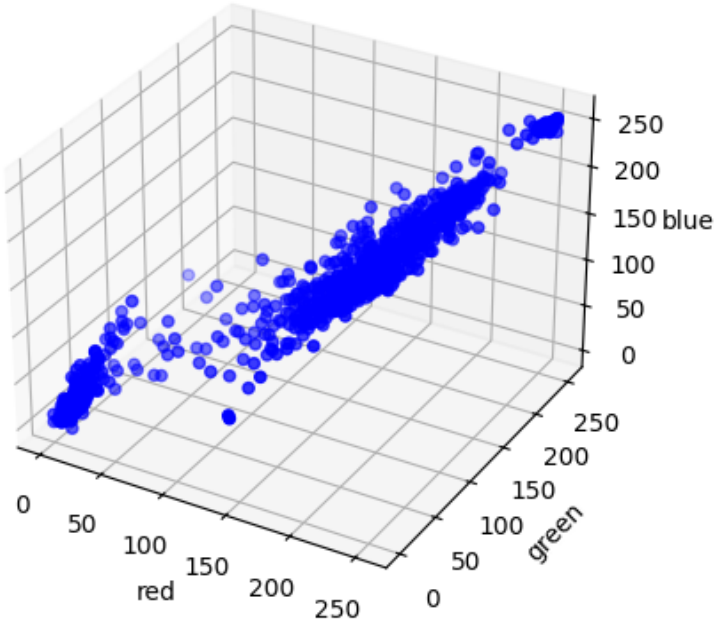
Comme sortie on a :

```
PS C:\Users\fheli\OneDrive\Documentos\3A_ENSTA\Machine Learning\Machine_Learning\Ex_06> & C:\Users\fheli\AppData\Local\Programs\Python\Python31
1\python.exe "c:/Users/fheli/OneDrive/Documentos/3A_ENSTA/Machine Learning/Machine_Learning/Ex_06/preprocessing.py"
Nombre de données d'apprentissage :  1692
```

5 Première analyse des données

Question 5.1: Dans preprocessing.py , visualiser en 2D et 3D les valeurs des descripteurs. La visu alisation en 2D des vecteurs d'apprentissage se fait par la fonction displayFeatures2d(feat) et en 3D displayFeatures3d(feat) .





Question 5.2: Décrivez explicitement ces graphiques en expliquant ce qu'ils représentent, en donnant leurs caractéristiques pour les histogrammes et les nuages de points (différents groupes? à quelles informations peut-on les relier?)

- Les histogrammes sur la diagonale principale montrent qu'il y a plein des pixels avec des valeurs proches de 0-20 d'intensité et 150-250 d'intensité, dans ce façon est possible affirmer que ces valeus plus basses sont liées à l'eau et les valeurs plus haute ont un fort liason avec terre/sable. Les nuages hors diagonale montrent une forte correlation linéaire entre les trois canaux. L'image 3D affichent qu'on a une certaine dvision de deux groupes de pixels, les pixels avec des valeurs plus basses et les pixels avec valeurs plus haut, comme ça est possible de diviser les pixels en deux groupes.

6 Approche non supervisée par la méthode des k-means

Question 6.1: Décrire avec vos mots la méthode des centres mobiles.

- La méthode des centres mobiles fonctionne de façon non supervisé c'est à dire que la méthode apprend à classifier les points en K groupes automatiquement.

L'algorithme place les centres initialement au hasard et les points son classés d'accord le centre plus proche. A chaque interaction les centres sont ajustés au milleur géometrique de leurs groupes respectifs. Ce processus est répeté jusqu'à les groupes soient stables et ne bougent plus.

Question 6.2: Appeler et compléter le script unsupervisedTraining.py qui permet de réaliser un apprentissage non supervisé du modèle de classement (classifieur) à l'aide de la fonction KMeans() du package scikit-learn (puissant package de Machine Learning pour python). Comment utilise-t-on cette fonction ? Quels en sont les paramètres de contrôle importants ?

```
def unsupervisedTraining(featLearn, method='kmeans'):
    ...
    if method == 'kmeans':

        Xs= StandardScaler().fit_transform(featLearn) #
           normaliserlesdescripteurs
        #---K-means
           ----------------------------------------------------------

        km= KMeans(n_clusters=nbCluster,init="k-means++",
           n_init=10, random_state=0).fit(Xs)
        model = km.fit(Xs)
        labels_km =km.labels_ #affectationdure
        J = km.inertia_ #fonctiondecoutJ
        s = silhouette_score(Xs,labels_km) #qualiteinterne
```

En regardant la documentation du package les paramètres de contrôle importants à configurer lors de l'instanciation sont :

- `n_clusters` : Définit le nombre de classes (K) à identifier dans les données.
- `init` : Détermine la méthode de placement initial des centres, comme `k-means++` qui les répartit de manière optimale pour accélérer la convergence et éviter les minima locaux.
- `n_init` : Fixe le nombre d'exécutions complètes de l'algorithme avec des points de départ différents, afin de conserver automatiquement le modèle minimisant la fonction de coût (l'inertie).
- `max_iter` : Limite le nombre maximal de déplacements des centres lors d'une exécution pour éviter une boucle infinie si le modèle peine à converger.
- `random_state` : Fixe la graine du générateur de nombres aléatoires pour assurer la stricte reproductibilité des résultats à chaque lancement du script.

Question 6.3: Compléter aralsea_main.py pour prédire les labels sur la base d'apprentissage grâce au modèle appris.

```
    ...

    # Apprentissage de la fonction de classement

    # ------------------------------------------------
    # YOUR CODE HERE

    model = unsupervisedTraining(featLearn, method='kmeans')

    # ------------------------------------------------
    # ------------------------------------------------




    # prediction des labels sur la base d'apprentissage

    # ------------------------------------------------
    # YOUR CODE HERE

    labelsLearn = model.predict(featLearn)

    # ------------------------------------------------
    # ------------------------------------------------
```

Question 6.4: Il est maintenant possible de visualiser les valeurs des descripteurs d'apprentissage et leur appartenance à l'une des classes (un des clusters). Nous utiliserons la fonction : displayFeatures2d et displayFeatures3d . Compléter aralsea_main.py . Faire varier le paramétrage de la fonction KMeans et analyser les différences et la qualité de l'apprentissage.

C'était possible de varié les paramètres de la fonction pour identifier les meilleurs paramètres pour la méthode, en-dessous on voit la sortie pour chaque paramètre testé :

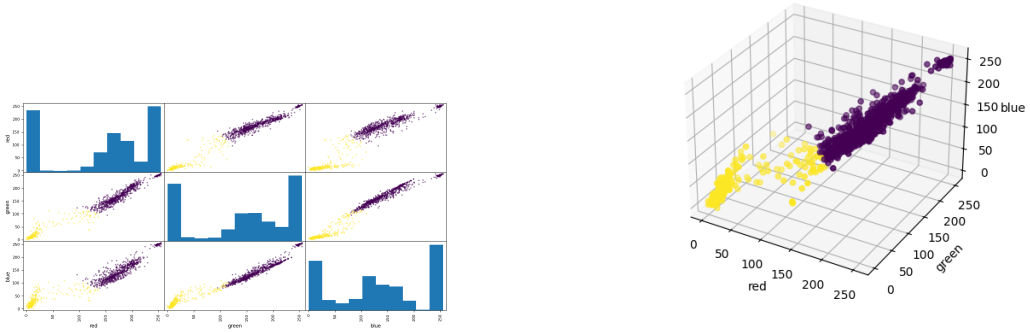


Figura 1: résultats pour k = 2.

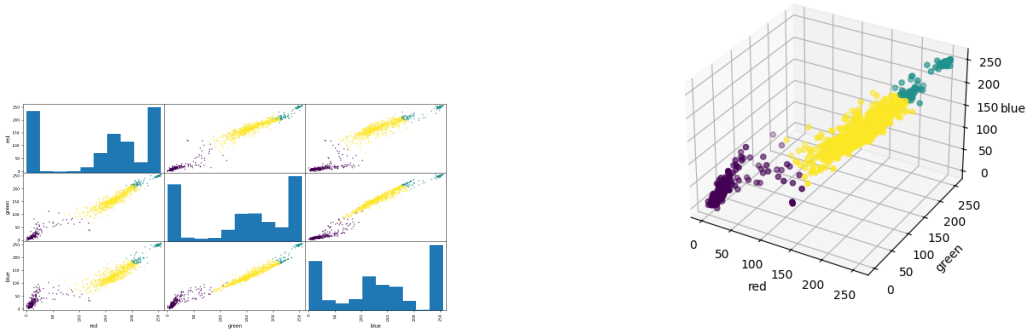


Figura 2: résultats pour k = 3.

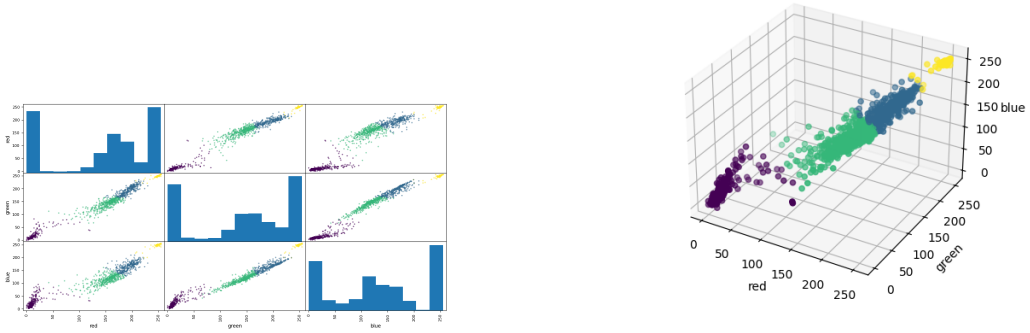


Figura 3: résultats pour k = 4.

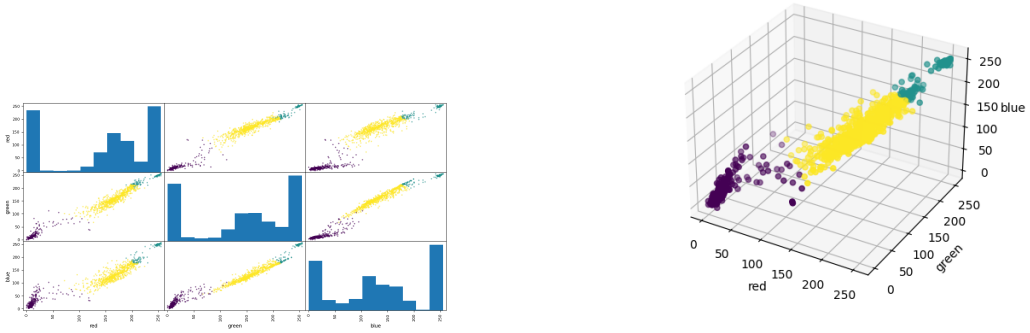


Figura 4: résultats en remplaçant la méthode k-means++ par random.

On peut rémarquer que le résultat avec k = 3 et la méthode k-means++ est le plus performant (figure 2). Dans ce cas c'est possible d'isoler les marges blanches de l'image dans un groupe, en plus identifier l'eau, avec les pixels des valeurs plus basse, et la terre.

Question 6.5: A ce stade, vous avez choisi les hyper-paramètres de vos méthodes de machine learning et obtenu le modèle de classement, nous pouvons alors utiliser les données d'apprentissage pour classifier l'ensemble des deux images. Il faut calculer les descripteurs sur toute l'image (mise sous la forme d'une matrice) puis utiliser le modèle de classement issu de l'apprentissage du k-means). Compléter les scripts aralsea_main.py et unsupervisedClassifying.py .

```
    ...
#%% Classement et estimation de la diminution de surface
# Classifying / Predicting / Testing

# mise en forme de l'image de 1973 et 1987 en matrice Num
   Pixels / Val Pixels

# ------------------------------------------------
# YOUR CODE HERE

h73, w73, c73 = img73.shape
h87, w87, c87 = img87.shape

feat73 = img73.reshape(h73*w73, c73)
feat87 = img87.reshape(h87*w87, c87)

# ------------------------------------------------
# ------------------------------------------------
```

```
18
19
20 # Classement des deux jeux de donnees et visualisation des
       resultats en image
21
22 # ------------------------------------------------
23 # YOUR CODE HERE
24
25 labels73 = unsupervisedClassifying(model, feat73)
26 labels87 = unsupervisedClassifying(model, feat87)
```

```
1     #%% package
2 import numpy as np
3 from sklearn.cluster import KMeans
4
5
6 #%% def unsupervisedClassifying
7 def unsupervisedClassifying(model, feat):
8
9     label = model.predict(feat)
10
11    return label
```

Question 6.6: Pour visualiser le résultat de classification, il faut que chaque classe prédite puisse être affectée au pixel correspondant de l'image. Cette image est calculée et visualisée par la fonction displayImageLabel.py . Compléter aralsea_main.py